

Engineering Specification: GBA Save State Interoperability Utility

Target Platform: Anbernic RG34XXSP (MuOS)
Host System: macOS (Development & Testing)

1. Hardware & OS Environment

1.1 Target Device: Anbernic RG34XXSP

Unlike its predecessors, the RG34XXSP features a distinct hardware profile that significantly benefits application development, particularly regarding memory overhead.

Component	Specification	Dev Implication
SoC	Allwinner H700 (Quad-core A53)	High performance for scripting; negligible latency.
RAM	2GB LPDDR4	Massive headroom. Python scripts can load entire save files into memory without risk of OOM (Out of Memory) kills.
Display	3.4-inch, 720 x 480	Aspect Ratio 3:2. This is wider than the standard 640x480 (4:3). TUI layouts must be adjusted to utilize the extra horizontal real estate (approx. 90 columns vs 80).
Input	D-Pad, Buttons	No touch. Navigation must be strictly keyboard-driven via whiptail.
Sleep	Hall Magnetic Switch	Lid close triggers sleep. File I/O must be atomic to prevent corruption if the lid is closed mid-write.

1.2 Operating System: MuOS (Linux)

MuOS on the H700 chipset is a lightweight Linux distribution. It uses a specific directory structure for user applications.

- **Python:** Python 3 is pre-installed at `/usr/bin/python3`.
- **UI Library:** whiptail (libnewt) is the native tool for menus and dialogs.
- **Mount Points:**
 - SD Slot 1: `/mnt/mmc` (System & Apps)
 - SD Slot 2: `/mnt/sdcard` (Roms & Saves)

2. Core Logic: The mGBA vs. VBA Conflict

The core utility function addresses the binary incompatibility between mGBA (default on MuOS) and VBA (standard on PC/Mac).

- **Standard Save (VBA/Flashcart):** Exactly 131,072 bytes (128KB) for most Pokemon games.
- **mGBA Save:** Appends a **16-byte RTC footer** containing timestamp data. Total size: 131,088 bytes.

Conversion Algorithm:

1. **Detection:** If file size is 131,088 bytes -> Identify as **mGBA**.
2. **Sanitization:** Truncate the last 16 bytes.
3. **Result:** A standard 131,072 byte file compatible with VBA.
4. **Note:** Converting VBA -> mGBA is usually not required; mGBA will accept a raw save and append a new footer upon next save. The critical path is **mGBA -> VBA**.

3. System Architecture

The application consists of a shell script entry point (for the UI) and a Python script (for the logic).

3.1 Directory Structure

MuOS requires a specific folder structure to recognize the app and its assets.

```
/mnt/mmc/MUOS/application/GBA_Fixer/  
├── launch.sh # Entry point & UI Loop  
├── convert.py # Binary processing logic  
├── assets/  
└── icon.png # 320x320px (Displayed in menu)
```

└─ preview.png # 640x480px (Displayed as background)

3.2 launch.sh (The UI Controller)

This script manages the whiptail interface. It must handle the file selection dialog.

- **Resolution Handling:** With a 720px width, whiptail dialogs should be set to **~85-90 characters wide** to look balanced.
- **File Browser:** Since whiptail does not have a built-in file picker like dialog's `--fselect`, we must implement a loop that lists `.sav` files in a menu.

4. Development & Testing Strategy (macOS + Docker)

Since you are developing on a Mac, you cannot run the MuOS shell scripts directly due to differences in `sed`, `awk`, and file paths. We will use a **Hybrid Testing Approach**.

4.1 Option A: Docker Container (Recommended)

This allows you to run the exact Linux shell environment on your Mac. There is no "Official MuOS Container," but a standard Debian slim image is 99% identical for this purpose.

Dockerfile:

Save this as Dockerfile in your project root.

Dockerfile

```
FROM python:3.11-slim-bookworm
```

```
# Install whiptail and basic utils to mimic MuOS environment
```

```
RUN apt-get update && apt-get install -y whiptail coreutils
```

```
# Create the MuOS directory structure mock
```

```
RUN mkdir -p /mnt/mmc/MUOS/application/GBA_Fixer \
&& mkdir -p /mnt/sdcard/ROMS/GBA
```

```
# Set working directory
```

```
WORKDIR /mnt/mmc/MUOS/application/GBA_Fixer
```

```
# Copy your local scripts into the container
```

```
COPY..
```

```
# Make scripts executable
RUN chmod +x launch.sh
```

```
CMD ["/bin/bash"]
```

Testing Workflow:

1. Build: `docker build -t muos-test`.
2. Run: `docker run -it --rm -v $(pwd):/mnt/mmc/MUOS/application/GBA_Fixer muos-test /bin/bash`
3. Inside Docker: Run `./launch.sh`. You will see the TUI inside your Mac terminal exactly as it will appear on the device.

4.2 Option B: Local macOS Testing

If you prefer not to use Docker, you must install the dependencies via Homebrew.

1. **Install Whiptail:** `brew install newt`
2. **Mock Paths:** Your Python script must detect if it's running on macOS and switch paths.

Python

```
import platform
import os
```

```
IS_MACOS = platform.system() == 'Darwin'
```

```
if IS_MACOS:
    # Create a local 'mock' folder for testing
    ROOT_DIR = os.path.join(os.getcwd(), "mock_sd")
else:
    # Actual MuOS paths
    ROOT_DIR = "/mnt/sdcard"
```

5. Implementation Logic (For Claude Code)

Pass the following instructions to the coding agent to generate the implementation.

5.1 convert.py Requirements

- **Input:** Accepts a single filename as a command-line argument.
- **Validation:**
 - Check if file exists.
 - Check size.
 - If 131,088 bytes: Perform **Trim** (Remove last 16 bytes).
 - If 131,072 bytes: Return "**Already Raw**".
 - Other sizes: Return "**Unknown Format**".
- **Safety:**
 - Create a .bak copy before touching the original.
 - Use `os.fsync()` to ensure write persistence.

5.2 launch.sh Requirements

- **Header:** Include MuOS metadata comments.
Bash
#!/bin/sh
NAME: GBA Save Fixer
ICON: icon.png
CATEGORY: Utility
- **Resolution Config:** Explicitly define whiptail dimensions for 720x480.
 - Height: 20 rows
 - Width: 85 cols
- **Logic Loop:**
 1. Scan `/mnt/sdcard/ROMS/GBA` (and `/mnt/mmc/ROMS/GBA`) for .sav files.
 2. Format the list for whiptail `--menu`.
 3. On selection, call `python3 convert.py "filename"`.
 4. Capture python output and display it in a whiptail `--msgbox`.

5.3 Deployment

1. **Zip:** Create a zip file named `GBA_Fixer.zip` containing the `GBA_Fixer` folder.
2. **Install:**
 - **Manual:** Extract to `/mnt/mmc/MUOS/application/`.
 - **MuOS Archive Manager:** Place zip in Archive folder on SD card and install via on-device menu.